Schoolic — Bugs And Complete Audit Report

Document Version: 1.0.0

> **Target Release:** Schoolic Enterprise v1.0 Production
> **Audit Date:** October 2026
> **Scope:** End-to-End System Audit across Cloudflare Workers API (`services/api`), Neon PostgreSQL Database (`database/migrations`), Cloudflare Pages Web Frontend, and Cross-Platform Flutter Client (`apps/flutter_app`).

Executive Summary & Production Readiness Scorecard

While the Schoolic repository features an extensive architecture (65 database tables, 180+ REST endpoints, 106 Flutter automated tests, and 148 backend unit tests passing), **critical defects, architectural race conditions, and missing end-to-end user provisioning flows currently prevent this system from operating safely in a live production environment.**

If deployed to production today without the remediations specified in this report:

1. **Financial payments will experience race-condition crashes and duplicate receipt numbers** whenever a parent verifies a transaction simultaneously with Cashfree's webhook.
2. **Staff and parents created by school administrators cannot log in**, as they lack linked authentication accounts in Neon Auth / Better Auth.
3. **Staff members with roles such as Accountant, Admissions Officer, or Principal are locked out of the web/desktop portal** by a client-side routing guard.
4. **Any database error during admissions or student enrollment leaves permanently orphaned records** due to the complete absence of transactional rollbacks (`db.transaction`).
5. **The document upload endpoint exposes the application to Worker memory exhaustion and Stored Cross-Site Scripting (XSS)** attacks.

Production Readiness Scorecard

Area	Grade	Status	Critical Blockers
Financial & Payments	**C-**	❌ Blocked	Receipt generation race condition; hardcoded sandbox credentials in `wrangler.jsonc` .
Data Consistency & Transactions	**D**	❌ Blocked	0 database transactions across entire API; sequential N+1 attendance sync loop.
Authentication & Identity Lifecycle	**D+**	❌ Blocked	Disconnected user provisioning: staff created with `userId = null` ; bulk imports use fake IDs.
Client Routing & Authorization	**C**	❌ Blocked	Router guard locks out non-superadmin staff (`accountant` , `admissions_officer` , `principal`).
File Storage & Security	**C+**	⚠️ At Risk	No file size limits on R2 upload (Worker OOM); arbitrary MIME download (`inline` XSS).
Parent & Student Portal (UI)	**B-**	⚠️ Incomplete	Backend APIs exist for leaves, bus tracking, PTM, and chat, but Flutter UI lacks corresponding views.
External Gateways (SMS / Push)	**F**	❌ Mock Only	No Indian DLT SMS gateway; malformed Meta WhatsApp URL; no FCM/APNs push engine.

1. Exhaustive Bug Catalog

Bug #1: Financial Receipt Number Race Condition & Duplicate Key 500 Crash

- **Severity: P0 — Critical (Financial Loss & Data Corruption)**
- **Component:** `services/api`
- **File Location:** `services/api/src/repositories/school-repository.ts` and `L1888-L1895`
- **Schema Definition:** `services/api/src/db/schema.ts`

Root Cause Analysis

In both `verifyPaymentOrder` and `handleCashfreeWebhook`, receipt numbers are generated by calculating `count(*)` over the `fee_payments` table:

```
// services/api/src/repositories/school-repository.ts: lines 1772-1778 & 1888-1895
const year = new Date().getFullYear();
const countRes = await db
  .select({ count: sql<number>`count(*)::int` })
  .from(feePayments)
  .where(eq(feePayments.schoolId, user.schoolId));
const nextSeq = (countRes[0]?.count ?? 0) + 1;
const receiptNumber = `REC-${year}-${String(nextSeq).padStart(5, '0')}`;
```

In `services/api/src/db/schema.ts`:

```
unique().on(table.schoolId, table.receiptNumber)
```

When a parent completes a payment through Cashfree:

1. The Flutter client receives the Cashfree SDK completion callback and immediately issues `POST /api/v1/portal/fees/verify-order`.
2. Simultaneously, Cashfree's server dispatches an asynchronous webhook to `POST /api/v1/portal/fees/cashfree-webhook`.
3. Both requests run concurrently on separate serverless Cloudflare Worker isolates.
4. Both isolates execute the `SELECT count(*)` query concurrently and calculate the exact same sequence number (e.g., `42`, generating `REC-2026-00042`).
5. Isolate A inserts the row successfully.
6. Isolate B executes `INSERT INTO fee_payments` with the identical `(schoolId, receiptNumber)`.
7. PostgreSQL triggers a unique constraint violation (`23505`).
8. If the user's browser/app is Isolate B, the parent receives an HTTP 500 failure screen indicating payment failure, **even though the bank account has already been debited**.
9. If Isolate B is the webhook, Cashfree receives an HTTP 500 error and schedules repeated webhook retries.

Production Fix

1. Create a dedicated PostgreSQL sequence or atomic counter table locked with `FOR UPDATE`:

```
CREATE SEQUENCE IF NOT EXISTS fee_receipt_seq;
```

2. Or use an atomic sequence query in Drizzle:

```
const [seqRow] = await tx.execute(sql`SELECT nextval('fee_receipt_seq') AS num`);
const receiptNumber = `REC-${year}-${String(seqRow.num).padStart(6, '0')}`;
```

Bug #2: Complete Absence of Database Transactions Across Entire Backend

- **Severity: P0 — Critical (Data Integrity & Orphaned Records)**
- **Component:** `services/api`
- **File Location:** Across all repository methods in `services/api/src/repositories/school-repository.ts`

Root Cause Analysis

A repository-wide search for `.transaction(` yields **0 occurrences**. Every complex multi-step mutation is executed as a series of disconnected, sequential SQL queries over HTTP.

Evidence & Scenarios:

1. **Applicant Enrollment (`school-repository.ts:L2322-L2355`):**
   - Step 1: Inserts new record into `students` .
   - Step 2: Inserts new record into `studentEnrollments` .
   - Step 3: Updates `admissionApplications` setting status to `enrolled` .
   - *Failure point:* If Step 2 fails (e.g., class capacity exceeded, constraint error, foreign key violation), the student record in Step 1 remains permanently committed. The student exists in the database but has no class, section, or academic year enrollment.
2. **Fee Payment Verification (`school-repository.ts:L1780-L1825`):**
   - Step 1: Updates `paymentOrders` to `PAID` .
   - Step 2: Inserts record into `feePayments` .
   - Step 3: Updates `studentFeeAllocations` to `paid` or `partial` .
   - *Failure point:* If Step 2 fails on receipt collision, Step 1 has already marked the order `PAID` , but no `feePayments` record exists and the student's allocation balance is not reduced.
3. **School Bootstrap (`services/api/src/routes/bootstrap.ts:L50-L80`):**
   - Step 1: Inserts `schools` .
   - Step 2: Inserts `appUsers` .
   - Step 3: Inserts `userRoles` .
   - *Failure point:* If Step 3 fails, the school and admin user exist, but the admin user has zero roles and cannot access any protected endpoint.

Production Fix

Wrap all multi-table mutations in Neon HTTP transactions:

```
await db.transaction(async (tx) => {
  const [student] = await tx.insert(students).values({...}).returning();
  await tx.insert(studentEnrollments).values({...});
  await tx.update(admissionApplications).set({...}).where(eq(...));
});
```

Bug #3: Admission Number Random Collision via Uniform Distribution

- **Severity: P1 — High (Operational Failure During Admissions)**
- **Component:** `services/api`
- **File Location:** `services/api/src/repositories/school-repository.ts`

Root Cause Analysis

When an applicant is approved and enrolled without a manual admission number, the system generates:

```
const admissionNumber =
  input.admissionNumber ||
  `ADM-${new Date().getFullYear()}-${Math.floor(1000 + Math.random() * 9000)}`;
```

The pool of possible numbers is only $9,000$ integers (1000 to 9999). Under the **Birthday Paradox**, the probability of a collision is: $$P(\text{collision}) \approx 1 - e^{-\frac{n^2}{2 \times 9000}}$$

- After **50 enrollments**, the collision probability is **13.0%**.
- After **118 enrollments**, the collision probability exceeds **53.8%**!
- After **200 enrollments**, the collision probability reaches **89.1%**.

Because `students` has a unique index on `(school_id, admission_number)`, whenever a collision occurs, the query crashes with a PostgreSQL unique constraint error, blocking the admissions officer from completing the enrollment.

Production Fix

Use an atomic database sequence per school or a monotonic counter:

```
const [seq] = await tx.execute(
  sql`SELECT coalesce(max(substring(admission_number from '[0-9]+$')::int), 0) + 1 AS next_val
      FROM students WHERE school_id = ${user.schoolId}`
);
const admissionNumber = `ADM-${year}-${String(seq.next_val).padStart(5, '0')}`;
```

Bug #4: Client-Side Router Guard Locks Out Institutional Staff

- **Severity: P1 — High (Access Control & Usability Defect)**
- **Component:** `apps/flutter_app`
- **File Location:** `apps/flutter_app/lib/src/core/router.dart`
- **Model Definition:** `apps/flutter_app/lib/src/core/models.dart`

Root Cause Analysis

In `router.dart`：

```
if (atLogin) {
  if (auth.session!.isAdmin) return '/admin';
  if (auth.session!.isTeacher) return '/teacher';
  return '/portal';
}
if (state.matchedLocation.startsWith('/admin') && !auth.session!.isAdmin) {
  return auth.session!.isTeacher ? '/teacher' : '/portal';
}
```

In `models.dart`:

```
bool get isAdmin => roles.contains(AppRole.admin);
bool get isTeacher => roles.contains(AppRole.teacher);
bool get isAccountant => roles.contains(AppRole.accountant);
bool get isAdmissionsOfficer => roles.contains(AppRole.admissionsOfficer);
bool get isPrincipal => roles.contains(AppRole.principal);
bool get isInstitutionalStaff =>
    isAdmin || isAccountant || isAdmissionsOfficer || isPrincipal;
```

When an **Accountant**, **Admissions Officer**, or **Principal** logs into the application:

1. `auth.session!.isAdmin` evaluates to `false`.
2. `atLogin` redirects them to `/portal` (the Student/Parent portal view).
3. If they attempt to navigate to `/admin` to manage fees or admissions, the router guard checks `!auth.session!.isAdmin`, catches them, and forcibly bounces them back to `/portal` or `/teacher`.
4. These staff members are **completely locked out of the administrative application**, even though the backend API explicitly authorizes them for these routes.

Production Fix

Change `auth.session!.isAdmin` to `auth.session!.isInstitutionalStaff` in `router.dart`:

```
if (atLogin) {
  if (auth.session!.isInstitutionalStaff) return '/admin';
  if (auth.session!.isTeacher) return '/teacher';
  return '/portal';
}
if (state.matchedLocation.startsWith('/admin') && !auth.session!.isInstitutionalStaff) {
  return auth.session!.isTeacher ? '/teacher' : '/portal';
}
```

Bug #5: Arbitrary File Upload & Stored Cross-Site Scripting (XSS) via Documents Bucket

- **Severity: P1 — High (Security Vulnerability & Resource Exhaustion)**
- **Component:** `services/api`
- **File Location:** `services/api/src/routes/admin.ts`

Root Cause Analysis

The document upload endpoint processes uploaded files as follows:

```
// admin.ts: lines 549-553
const raw = await c.req.arrayBuffer();
if (raw.byteLength === 0) {
  throw new ApiError(400, 'No file uploaded');
}
bytes = new Uint8Array(raw);
contentType = c.req.header('content-type') || contentType;
```

And on document download:

```
// admin.ts: lines 592-597
c.header('Content-Type', file.contentType);
c.header('Content-Length', String(file.contentLength));
c.header('Content-Disposition', `inline; filename="${doc.fileName || file.fileName}"`);
return c.body(file.body as any);
```

Vulnerabilities:

1. **Worker Memory Exhaustion (Denial of Service):** Cloudflare Workers have strict memory limits (128MB per isolate). Uploading a large file loads the entire payload into the Worker's memory buffer (`c.req.arrayBuffer()`), triggering an uncatchable isolate crash. There is **no maximum file size check** (e.g., 10MB limit).
2. **Stored Cross-Site Scripting (XSS):** There is **no MIME type validation**. An attacker can upload an HTML or SVG file containing malicious JavaScript (`<script>alert(document.cookie)</script>`). When a school administrator clicks the download link, the file is served with `Content-Disposition: inline` and `Content-Type: text/html` on the API domain. The attacker's script executes within the admin's session context, enabling session hijacking and token theft.

Production Fix

1. Enforce strict maximum file size (e.g., `10 * 1024 * 1024` bytes / 10MB) before buffering.
2. Restrict MIME types to an allowed whitelist: `application/pdf` , `image/png` , `image/jpeg` , `image/webp` .
3. Force download disposition: `Content-Disposition: attachment; filename="..."` .
4. Include security header: `X-Content-Type-Options: nosniff` .

Bug #6: In-Memory Rate Limiting Isolate Bypass & Edge Memory Leak

- **Severity: P2 — Medium (DoS Resilience & Edge Architecture)**
- **Component:** `services/api`
- **File Location:** `services/api/src/http/rate-limiter.ts`

Root Cause Analysis

```
const memoryRateLimitStore = new Map<string, RateLimitRecord>();
```

In a serverless edge environment like Cloudflare Workers:

1. Cloudflare routes requests across hundreds of global edge data centers (PoPs). Memory is **not shared** between data centers or between isolates in the same data center.
2. An attacker can bypass the rate limit simply by rotating IP addresses or distributing requests across locations.
3. Furthermore, `memoryRateLimitStore` has no background eviction timer. If an attacker sends requests with thousands of random pseudo-IPs in `X-Forwarded-For` , the Map grows unbounded until the isolate exhausts memory.

Production Fix

Migrate the rate limiter to use Cloudflare's native Rate Limiting binding (`c.env.RATE_LIMITER`), Cloudflare KV with TTL, or Neon database-backed sliding counters.

Bug #7: Malformed Meta WhatsApp Cloud API Endpoint

- **Severity: P2 — Medium (Integration Failure)**
- **Component:** `services/api`
- **File Location:** `services/api/src/notifications/notification-service.ts`

Root Cause Analysis

The WhatsApp dispatch logic calls:

```
const response = await fetch('https://graph.facebook.com/v21.0/messages', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${whatsappToken}`,
    'Content-Type': 'application/json',
  },
  ...
```

Meta's WhatsApp Business Cloud API requires the **Phone Number ID** in the URL path: `https://graph.facebook.com/v21.0/{PHONE_NUMBER_ID}/messages` Calling `v21.0/messages` without the ID returns an HTTP 400 or 404 error from Meta Graph API.

Production Fix

1. Add `WHATSAPP_PHONE_NUMBER_ID` to Worker environment variables.
2. Construct the URL dynamically:

```
const phoneId = (this.env as any)?.WHATSAPP_PHONE_NUMBER_ID;
const response = await fetch(`https://graph.facebook.com/v21.0/${phoneId}/messages`, { ... });
```

Bug #8: Deprecated `dart:html` Imports Incompatible with Flutter WebAssembly (WASM)

- **Severity: P2 — Medium (Web Modernization & Build Toolchain)**
- **Component:** `apps/flutter_app`
- **File Location:**
  - `apps/flutter_app/lib/src/core/csv_exporter_web.dart`
  - `apps/flutter_app/lib/src/core/auth_transport_web.dart`

Root Cause Analysis

Both files import `dart:html` :

```
import 'dart:html' as html;
import 'dart:html';
```

In Flutter 3.22+, Google introduced WebAssembly (`wasm`) compilation as the official high-performance standard for Flutter Web. The legacy `dart:html` library is incompatible with `dart2wasm` and will cause compilation to fail when executing `flutter build web --wasm` .

Production Fix

Refactor both web helper files to use `package:web` and `dart:js_interop` :

```
import 'package:web/web.dart' as web;
import 'dart:js_interop';
```

Bug #9: GoRouter Lacks Public Routes for Admissions and Certificate Verification

- **Severity: P2 — Medium (Broken External User Journey)**
- **Component:** `apps/flutter_app`
- **File Location:** `apps/flutter_app/lib/src/core/router.dart`

Root Cause Analysis

The GoRouter definition only registers `/login` , `/admin` , `/teacher` , and `/portal` . Any unauthenticated access to paths like:

- `/apply/:schoolCode` (Public admission application form)
- `/verify/:verificationCode` (Public QR code certificate verification) is intercepted by line 22:

```
if (!signedIn) return atLogin ? null : '/login';
```

External prospective parents or employers scanning a QR code on a printed transfer certificate are forcibly redirected to the staff login screen!

Production Fix

Add public unauthenticated routes to GoRouter before the auth guard check:

```
final isPublicRoute = state.matchedLocation.startsWith('/apply') ||
                      state.matchedLocation.startsWith('/verify');
if (isPublicRoute) return null;
```

2. Missing End-to-End Backend Implementations

Gap #1: Staff & Parent User Account Provisioning & Invitation Pipeline

- **Current State:**
  - `POST /api/v1/admin/staff` inserts a record into the `staff` table, leaving `userId` as `null` (`school-repository.ts:L758-L761`).
  - Bulk staff import generates random placeholder strings: `neonUserId: usr_${Math.random()...}` (`school-repository.ts:L3201`).
  - Bulk student import generates fake parent IDs: `neonUserId: parent_${Math.random()...}` (`school-repository.ts:L3316`).

    - The authentication middleware (`services/api/src/http/auth.ts:L215-L219`) queries `app_users` matching `neon_user_id = sessionUser.id` .
- **The Failure:** None of these users exist in Neon Auth / Better Auth. If an imported teacher or parent attempts to sign up or log in, `auth.ts` throws `403: User is not provisioned for this pilot` . They can **never** log in.
- **Required Implementation:**
    i. Create an admin user provisioning endpoint or webhook listener.
    ii. Integrate with Neon Auth / Better Auth Admin API (`/api/auth/admin/users`) to create genuine auth identities with temporary credentials or magic link invitations.
    iii. Send an onboarding email with an invitation link.
    iv. Link the resulting `neon_user_id` to `app_users` and `staff.user_id` .

Gap #2: Dynamic Guardian Account Creation & Linking in Administrative UI

- **Current State:**
    - The API has an endpoint: `POST /api/v1/admin/students/guardian-links` (`createStudentGuardianLinkSchema`).
    - However, this endpoint is **never called anywhere in the Flutter client code**.
    - When an admin manually adds a student via `AddStudentDialog` , only `students` and `student_enrollments` records are created. No guardian record or link is created.
- **The Failure:** When a parent logs into the Parent Portal, `portalStudentsFor` queries `student_guardians WHERE app_user_id = user.appUserId` . Because no guardian link was created, the portal displays: `"No Student Profile Linked"` .
- **Required Implementation:**
    i. In the Flutter `AddStudentDialog` , add input fields for Parent/Guardian Email & Phone.
    ii. In the backend `createStudent` flow, automatically create an `app_users` record for the parent (with role `parent`) and an entry in `student_guardians` within the same database transaction.

Gap #3: Attendance Sync Batch Upsert Optimization (N+1 Query Bottleneck)

- **Current State:**
    - In `syncOfflineAttendance` (`school-repository.ts:L6430-L6460`), the repository loops through every student record in every batch sequentially.
    - For each student record, it executes a `SELECT ... FROM attendance_records` followed by an `UPDATE` or `INSERT` .
- **The Failure:** Syncing 200 student attendance records generates **400 sequential HTTP database queries** from Cloudflare Workers to Neon. At 40ms per round-trip, this operation takes **16 seconds**, risking Cloudflare Worker execution timeouts (HTTP 524).
- **Required Implementation:** Replace the sequential loop with a single multi-row `INSERT ... ON CONFLICT DO UPDATE` :

```
await db
  .insert(attendanceRecords)
  .values(batchRecords)
  .onConflictDoUpdate({
    target: [attendanceRecords.schoolId, attendanceRecords.sessionId, attendanceRecords.studentId],
    set: {
      status: sql`excluded.status`,
      remarks: sql`excluded.remarks`,
      updatedAt: new Date(),
    },
  });
```

Gap #4: Production External SMS Gateway (Indian DLT) & Mobile Push Notification Engine

- **Current State:**
  - `NotificationService` (`services/api/src/notifications/notification-service.ts`) contains only sandbox simulation fallbacks for SMS and Push channels.
- **The Failure:** In Indian schools, parents rely heavily on SMS (due to varied smartphone accessibility). Under Telecom Regulatory Authority of India (TRAI) regulations, all SMS traffic must comply with Distributed Ledger Technology (DLT) template registration. The application cannot deliver real absence alerts or fee reminders.
- **Required Implementation:**
  i. Integrate an Indian SMS provider (e.g., **MSG91** or **Kaleyra**) with DLT Entity ID and approved Template IDs.
  ii. Implement Firebase Cloud Messaging (FCM) v1 HTTP API to dispatch mobile push notifications to Android and iOS devices.

Gap #5: Direct Presigned Storage Uploads via Cloudflare R2

- **Current State:**
  - Admission document files are buffered in the Cloudflare Worker isolate's memory (`c.req.arrayBuffer()`) and streamed to R2 via `c.env.DOCUMENTS_BUCKET.put()` .
- **The Failure:** Large document uploads (e.g., high-resolution medical records, portfolios) risk hitting the 128MB Worker memory limit, while tie-up Worker CPU time.
- **Required Implementation:** Implement S3-compatible presigned URL generation (`@aws-sdk/s3-request-presigner` or Workers R2 signed URL binding):
  i. Client calls `POST /api/v1/admin/admissions/documents/:id/upload-url` .
  ii. Worker returns a short-lived presigned PUT URL.
  iii. Client uploads the binary directly to R2.
  iv. Client notifies the API upon completion to record metadata.

Gap #6: Production Cashfree Configuration & Webhook Verification

- **Current State:**
  - In `services/api/wrangler.jsonc:L69-L70` , the `production` environment target is configured with:

```
"CASHFREE_APP_ID": "TEST112097243d3d637f11e796a7739842790211",
"CASHFREE_ENV": "sandbox"
```

- **The Failure:** Deploying the API to production as configured will attempt to process real tuition payments against Cashfree's sandbox gateway.
- **Required Implementation:**
  i. Configure production Cashfree App ID and Secret via encrypted Cloudflare Worker secrets (`wrangler secret put CASHFREE_SECRET_KEY --env production`).
  ii. Set `CASHFREE_ENV` to `production` .
  iii. Ensure Cashfree signature validation (`verifyWebhookSignature`) is strictly enforced in production with zero test bypasses.

Gap #7: DPDP (Digital Personal Data Protection) Data Export Execution Worker

- **Current State:**
  - `POST /api/v1/portal/dsr/requests` records a data subject request in `dsr_requests` .
- **The Failure:** There is no asynchronous background processor to extract the student's personal data, academic records, attendance history, and fee receipts into an encrypted, downloadable ZIP bundle to comply with statutory DPDP turnaround windows.

- **Required Implementation:** A scheduled worker or queue consumer that gathers user data, packages it as JSON/CSV into a temporary R2 storage bucket, and emails a secure, time-limited download link.

3. Frontend-to-Backend Wiring Gaps (Missing Views)

While the backend implements comprehensive REST endpoints for the following features, the corresponding user interfaces have not yet been built into the Flutter client:

Student App / Parent Portal Gaps

1. **Student Leave Application:**
   - *Backend Available:* `POST /api/v1/portal/leaves` (`createStudentLeaveSchema`).
   - *Missing UI:* No form in `StudentAppView` allowing parents to select dates, provide a medical reason, and submit a leave request.
2. **Live Transport & Bus Tracking:**
   - *Backend Available:* `GET /api/v1/portal/transport/track` .
   - *Missing UI:* No map view or stop timeline in `StudentAppView` showing assigned route, vehicle number, driver contact, and GPS status.
3. **Official Certificate Retrieval:**
   - *Backend Available:* `GET /api/v1/portal/certificates` .
   - *Missing UI:* No screen for parents to view issued Transfer Certificates (TC), Study Certificates, or Bonafide Certificates.
4. **Parent-Teacher Meeting (PTM) Booking:**
   - *Backend Available:* `GET /api/v1/portal/ptm/slots` , `POST /api/v1/portal/ptm/slots/:id/book` .
   - *Missing UI:* No calendar interface for parents to view available time slots with their child's teachers and book meetings.
5. **Real-Time Teacher-Parent Messaging:**
   - *Backend Available:* `GET /api/v1/portal/chat/conversations` , `POST /messages` .
   - *Missing UI:* The portal has no chat conversation screen.

Teacher Dashboard Gaps

1. **Teacher Leave Management:**
   - *Backend Available:* `GET /api/v1/teacher/my-leaves` , `POST /api/v1/teacher/my-leaves` .
   - *Missing UI:* Teachers have no drawer tab to apply for casual/sick leave or view remaining leave balances.
2. **Student Leave Approval:**
   - *Backend Available:* `GET /api/v1/teacher/student-leaves` , `PATCH /student-leaves/:id` .
   - *Missing UI:* Class teachers have no interface to review and approve/reject leave applications submitted by parents of their class.

Public Unauthenticated Routes

1. **Public Admissions Application:**
   - *Backend Available:* `POST /api/v1/public/admissions/apply` .
   - *Missing UI:* No standalone web landing page (`/apply/:schoolCode`) for prospective parents.
2. **Public Certificate Verification:**
   - *Backend Available:* `GET /api/v1/public/certificates/verify/:code` .
   - *Missing UI:* No verification landing page (`/verify/:code`) for scanning printed document QR codes.

4. Production Hardening & Operational Checklist

Item	Status	Action Required
Atomic Receipt Sequences	❌ Missing	Create PostgreSQL sequence for receipt generation; remove `SELECT count(*)` logic.
Database Transactions	❌ Missing	Wrap all multi-entity mutations in `db.transaction(...)`.
Role Guard Correction	❌ Missing	Update `router.dart` to use `isInstitutionalStaff`.
R2 Upload Restrictions	❌ Missing	Enforce 10MB limit; whitelist MIME types; set `Content-Disposition: attachment`.
Edge Rate Limiting	⚠️ In-Memory	Replace in-memory Map with Cloudflare Rate Limiting binding or Cloudflare KV.
Indian DLT SMS Gateway	❌ Missing	Implement MSG91 / Kaleyra integration with DLT Template IDs.
Attendance Batch Upsert	⚠️ N+1 Loop	Convert sequential loop to single `INSERT ... ON CONFLICT DO UPDATE`.
CORS Whitelist Configuration	⚠️ Hardcoded	Ensure production Worker environment variable supports custom school domains.
Production Cashfree Secrets	⚠️ Sandbox	Configure live production keys and set `CASHFREE_ENV: "production"`.
WebAssembly Compatibility	⚠️ Deprecated	Replace `dart:html` with `package:web` and `dart:js_interop`.
Public Route Registration	❌ Missing	Register `/apply/:code` and `/verify/:code` as unauthenticated paths in `GoRouter`.

5. Prioritized Remediation Plan

```
Mermaid render error: Parse error on line 2:
... Critical Bug Fixes (Days 1-2)        P
----------------------^
Expecting 'SEMI', 'NEWLINE', 'SPACE', 'EOF', 'GRAPH', 'DIR', 'subgraph', 'SQS', 'end', 'AMP', 'COLON', 'START_LINK',
'STYLE', 'LINKSTYLE', 'CLASSDEF', 'CLASS', 'CLICK', 'DOWN', 'UP', 'NUM', 'NODE_STRING', 'BRKT', 'MINUS', 'MULT',
'UNICODE_TEXT', got 'PS'
```

Remediation Schedule:

Phase 1: Immediate Critical Fixes (Within 48 Hours)

1. **Fix Receipt Number Race Condition:** Implement atomic sequence or lock-based numbering in `verifyPaymentOrder` and `handleCashfreeWebhook`.
2. **Add Transaction Rollbacks:** Wrap `enrollApplicant`, `verifyPaymentOrder`, `createStudent`, and `bootstrap` in `db.transaction`.
3. **Correct Router Guard:** Update `apps/flutter_app/lib/src/core/router.dart` to grant access to `isInstitutionalStaff`.
4. **Harden Document Uploads:** Enforce 10MB ceiling, strict PDF/image MIME whitelist, and `Content-Disposition: attachment`.

Phase 2: User Onboarding & Performance (Within 5 Days)

1. **Auth Identity Sync:** Connect `POST /admin/staff` and bulk imports with Neon Auth / Better Auth user creation and email invitation delivery.
2. **Guardian Account Linking:** Wire `POST /admin/students/guardian-links` into the student creation modal.
3. **Batch Attendance Sync:** Refactor `syncOfflineAttendance` to a single batch upsert query.
4. **Fix WhatsApp Endpoint & Add SMS Gateway:** Correct Meta Graph path and integrate Indian DLT SMS provider.

Phase 3: UI Parity & Production Verification (Within 8 Days)

1. **Complete Student/Parent Views:** Add Leave Application, Bus Tracking, and PTM booking to `StudentAppView`.
2. **Complete Teacher Views:** Add My Leaves and Student Leave Review to `TeacherDashboard`.
3. **Add Public Routes:** Register `/apply/:schoolCode` and `/verify/:code` in `router.dart`.
4. **Production Configuration:** Deploy live Cashfree credentials and configure custom domain CORS origins.